

XAI-SDN: An explainable entropy-guided machine learning framework for real-time DDoS detection in software defined networks

Adeel Ahmad^a, Ali Akarma^a, Hasan Razzaqi^{b,*}, Toqeer Ali Syed^a, Hammad Muneer^c, Salman Jan^b

^aFaculty of Computer and Information Systems, Islamic University of Madinah, Madinah, Saudi Arabia

^bFaculty of Computer Studies, Arab Open University, Bahrain

^cDepartment of Computer Science, Islamia University of Bahawalpur, Bahawalpur, Pakistan

Abstract

Software Defined Networking (SDN) centralizes network control in a programmable controller, and this very centralization exposes a single point of failure: a successful Distributed Denial of Service (DDoS) attack against the controller or its control channel can render the entire network inoperable. Machine learning (ML) detectors achieve high accuracy on this task but typically operate as black boxes, which prevents security operators from validating and auditing individual detection decisions. This paper presents XAI-SDN, a lightweight, interpretable framework for real-time DDoS detection in SDN environments. XAI-SDN augments the 80 statistical flow features produced by CICFlowMeter with eight Shannon entropy metrics maintained by a constant-time $\mathcal{O}(1)$ rolling-window algorithm, classifies the resulting 88-dimensional flow representation with a Random Forest, and at-

*Corresponding author.

Email address: `hasan.razzaqi@aou.org.bh` (Hasan Razzaqi)

taches a per-prediction SHAP TreeExplainer attribution to every raised alert. On a fixed temporal split of the complete CIC-DDoS2019 SYN partition (3.59 million flows, no subsampling), XAI-SDN attains 99.9987% accuracy, 99.9621% macro F1, and an AUC-ROC of 1.0000, with only 14 misclassifications among 1,077,798 held-out flows. The pipeline sustains 0.0165 ms per flow (60,606 flows/s) without explanation generation and 0.5122 ms per flow (1,953 flows/s) with full SHAP support under the benchmark’s 99.14% DDoS prevalence. A multi-seed ablation with Wilcoxon signed-rank testing shows that the entropy and CICFlowMeter feature families are complementary, and an explicit robustness analysis quantifies how split methodology affects the achievable false positive rate, positioning XAI-SDN as a step toward balancing detection performance and operational transparency in next-generation SDN security.

Keywords: DDoS detection, software defined networks, explainable AI, Shannon entropy, random forest, SHAP, network security

1. Introduction

Software Defined Networking (SDN) has transformed the management of network infrastructure by decoupling the data plane from the control plane, enabling centralized, programmatic governance of the network through a dedicated controller [1, 2]. The paradigm is widely deployed in enterprise data centres and cloud infrastructure because it offers flexibility, global network visibility, and dynamic traffic engineering. Centralization, however, has a structural cost: the controller constitutes a single point of failure, and a successful Distributed Denial of Service (DDoS) attack that saturates the

controller or its control channel can render the entire network inoperable.

Modern DDoS campaigns combine volumetric flooding—UDP amplification, TCP SYN floods, and ICMP sweeps—with application-layer vectors such as HTTP floods and SlowLoris [3, 4, 5]. Machine learning (ML) has become the leading detection approach for SDN because learned classifiers generalize beyond the fixed patterns of signature-based systems. Yet the models deployed in practice are predominantly black boxes: they do not explain why a given flow was classified as malicious. In security operations this opacity is a concrete liability, because operators must validate, audit, and act on each detection decision, and emerging regulatory frameworks increasingly require that automated security decisions be traceable [6, 7, 8].

Three requirements must therefore be satisfied simultaneously by a practical SDN DDoS detector: (i) detection quality high enough to protect an infrastructure in which a single missed volumetric attack is catastrophic; (ii) per-flow processing cost low enough to keep pace with controller-scale flow arrival rates; and (iii) per-decision transparency sufficient for operator triage and audit. Existing work typically addresses at most two of the three: entropy-based statistical detectors are fast and interpretable but less accurate than learned models [9], deep detectors are accurate but opaque and computationally heavy, and post-hoc explanation of deep models is usually too slow for per-flow operation.

This paper presents XAI-SDN, a framework designed around all three requirements at once. XAI-SDN enriches the standard CICFlowMeter statistical feature set with eight Shannon entropy metrics maintained by a constant-time rolling algorithm, classifies flows with a Random Forest, and generates

an exact SHAP TreeExplainer attribution for every alert it raises. The design deliberately pairs an ensemble model whose accuracy is competitive with deep alternatives with an explanation method that is exact and fast for tree ensembles, so that transparency is obtained without sacrificing throughput.

The main contributions are as follows.

- **Entropy-guided feature augmentation.** An original feature augmentation approach that enriches the 80 CICFlowMeter flow features with eight interpretable Shannon entropy metrics computed over a sliding window, supported by an $\mathcal{O}(1)$ rolling update algorithm (Section 5) whose per-flow cost is independent of the window size.
- **Explainability-integrated detection pipeline.** A per-prediction pipeline coupling a Random Forest classifier with SHAP TreeExplainer so that every dispatched alert carries an exact feature attribution, enabling operator triage directly from the alert payload (Section 4).
- **Full-scale empirical evaluation.** A comprehensive evaluation on the entire SYN partition of CIC-DDoS2019—3.59 million flows without subsampling—under a leakage-free temporal split, complemented by a multi-seed ablation study with Wilcoxon signed-rank significance testing (Sections 6–8).
- **Honest operational characterization.** A corrected latency analysis that accounts for SHAP invocation under realistic DDoS prevalence, and an explicit false-positive-rate robustness analysis showing how split methodology changes the attainable operating point (Section 7), both of which are routinely omitted in prior work.

The remainder of the paper is organized as follows. Section 2 reviews related work. Section 3 defines the threat model and formalizes the detection problem. Section 4 describes the XAI-SDN architecture and Section 5 its mathematical foundation. Section 6 details the experimental setup. Section 7 reports detection, latency, and explainability results. Section 8 presents the ablation and baseline comparison. Section 9 discusses deployment considerations and limitations, and Section 10 concludes.

2. Related work

2.1. Entropy-based DDoS detection in SDN

Entropy is a natural measure of the distributional concentration that characterizes flooding behaviour [10]. In a seminal study, Lakhina et al. [11] showed that entropy summaries of traffic feature distributions expose a broad class of anomalies that volume-based metrics miss, and entropy-based techniques have since become a staple of the network anomaly detection literature [12]. Mousavi and St-Hilaire [9] showed that entropy-based early detection at the SDN controller substantially reduces attack damage, using a simple threshold on destination-address entropy. Lall et al. [13] established that entropy estimation over sliding windows of data streams is an efficient primitive for anomaly detection at line rate. These approaches are computationally attractive and directly interpretable, but a fixed entropy threshold cannot capture the interaction between distributional and per-flow statistical evidence, which limits detection quality relative to learned models. XAI-SDN continues this tradition but repositions entropy: rather than acting as the detector itself, $\mathcal{O}(1)$ rolling entropy becomes an early-stage feature genera-

tor, so that discriminative distributional statistics are available both to the classifier and to the SHAP explainer.

2.2. Machine learning for SDN DDoS detection

ML-based SDN DDoS detection has been studied extensively, with ensemble methods receiving particular attention for their accuracy, efficiency, and native feature-ranking capability. Braga et al. [14] provided an early demonstration that lightweight learning over OpenFlow statistics can detect flooding directly at the controller, and subsequent work explored progressively deeper architectures, including DNN-based flow classifiers [15] and stacked-autoencoder multi-vector DDoS detectors [16], which improve raw detection metrics at substantial computational and transparency cost. The CIC-IDS and CIC-DDoS benchmark families [17, 4] standardized flow-level evaluation through the CICFlowMeter feature extractor and made large-scale comparison possible; more recent SDN-specific datasets such as InSDN [18] extend this line to traffic captured on real SDN testbeds. Across this literature, tree ensembles such as Random Forest [19] and gradient boosting [20] consistently rank among the strongest performers at a small fraction of the inference cost of deep models, a pattern our baseline study (Section 8) reproduces. However, published evaluations frequently rely on random splits of temporally ordered captures, which leaks near-duplicate flows across the split boundary and overstates operational performance—a pitfall documented both by surveys of intrusion detection datasets [21] and by influential critiques of machine learning practice in security [22, 23]; Section 7 quantifies this effect explicitly.

2.3. Explainable AI for intrusion detection

Explainable AI (XAI) offers post-hoc and intrinsic interpretability mechanisms for security models [8]. Foundational work established both a taxonomy of black-box explanation methods [24] and the methodological requirements for evaluating interpretability claims rigorously [25]. Ribeiro et al. [26] introduced LIME, which fits local surrogate models around individual predictions; Lundberg and Lee [27] introduced SHAP, which unifies several existing explanation approaches under a single additive-attribution framework with consistency guarantees, and TreeSHAP [28] extends it to tree-based models with exact polynomial-time computation. Gaspar et al. [29] applied LIME and SHAP to a multi-layer-perceptron intrusion detector, demonstrating the operational value of attributions but paying the latency cost of model-agnostic explanation over a deep model. In security settings specifically, Warnecke et al. [30] showed that explanation methods differ sharply in fidelity, stability, and runtime, which motivates our choice of exact TreeSHAP over approximate model-agnostic explainers. Table 1 situates XAI-SDN in this landscape. To the best of our knowledge, the combination of $\mathcal{O}(1)$ entropy augmentation, Random Forest classification, and exact per-prediction TreeSHAP explanation, evaluated with a realistic characterization of explanation latency under actual DDoS prevalence, has not previously been demonstrated.

Table 1: Qualitative comparison of XAI-SDN with representative prior approaches.

Approach	Detection mechanism	Entropy features	Per-flow XAI	Explanation cost	SDN integration
Mousavi and St-Hilaire [9]	Entropy threshold	Destination IP only	Intrinsic	Negligible	Controller-embedded
Lall et al. [13]	Streaming entropy estim.	Stream sketches	Intrinsic	Negligible	Not addressed
Nychis et al. [10]	Entropy anomaly corr.	Multiple distributions	Intrinsic	Negligible	Not addressed
Gaspar et al. [29]	MLP classifier	None	Post-hoc LIME/SHAP	High (model-agnostic)	Not addressed
XAI-SDN (this work)	RF on 88-dim features	Eight $O(1)$ rolling	Exact TreeSHAP	0.5 ms per alert	Controller-embedded

3. Threat model and problem formulation

3.1. Threat model

We consider a volumetric DDoS adversary targeting an SDN deployment. The adversary controls a botnet of compromised hosts and launches TCP SYN flood traffic toward victims reachable through the SDN data plane, with the second-order objective of exhausting the controller through the table-miss path: each unmatched flow triggers a `packet_in` event, so a sufficiently high rate of spoofed flows saturates the control channel and controller CPU. The adversary may spoof source addresses and vary packet timing but does not control the SDN controller, the switches’ management interfaces, or the detection module itself; adaptive adversaries that shape traffic specifically to evade entropy features are out of scope and discussed as future work in Section 9. The defender observes flow statistics exported by OpenFlow switches and must decide, per flow and in real time, whether the flow belongs to an attack, and must justify that decision to a human operator.

3.2. Problem formulation

Let f_i denote the i -th flow observed at the controller and let $\mathbf{x}_i \in \mathbb{R}^d$ with $d = 88$ be its feature representation, composed of 80 CICFlowMeter statistical features and 8 windowed entropy features (Section 5). The detection

task is to learn a scoring function $g_\theta: \mathbb{R}^d \rightarrow [0, 1]$ and threshold τ such that the decision rule

$$\hat{y}_i = \mathbb{K}[g_\theta(\mathbf{x}_i) \geq \tau] \quad (1)$$

maximizes macro-averaged F1 under severe class imbalance, subject to a per-flow latency budget L covering feature construction, inference, and (for positive decisions) explanation generation. In addition to the label, the system must emit for every alert an attribution vector $\phi_i \in \mathbb{R}^d$ satisfying the additive efficiency property $g_\theta(\mathbf{x}_i) = \phi_0 + \sum_{j=1}^d \phi_{ij}$, so that the evidence behind each decision is decomposable over features and auditable by the operator.

4. XAI-SDN framework and architecture

XAI-SDN is built on three design principles: (i) computational efficiency, using $\mathcal{O}(1)$ rolling entropy so that feature augmentation never becomes the bottleneck in high-throughput SDN environments; (ii) transparency, producing actionable per-flow SHAP attributions rather than only global feature rankings; and (iii) robustness, combining statistical and information-theoretic views of traffic so that evidence of flooding is captured both per flow and across flows. The detection pipeline comprises five stages—flow aggregation, feature extraction and entropy augmentation, Random Forest classification, SHAP-based explanation generation, and alert dispatch to the security dashboard—as shown in Fig. 1. Flow statistics are retrieved from OpenFlow switches through the southbound API at a configurable polling interval of 500 ms.

Each flow record is processed by the XAI-SDN module, implemented on an OpenFlow-based SDN control plane [1] with scikit-learn [31]. The mod-

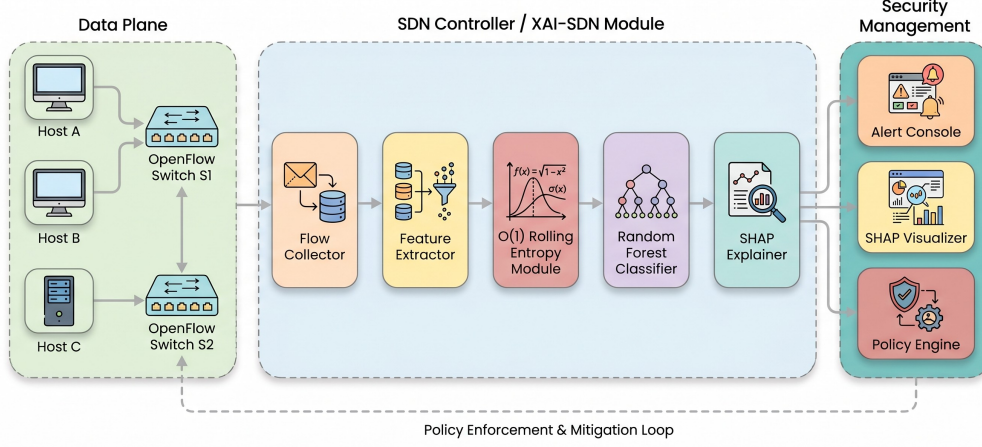


Figure 1: XAI-SDN system architecture. OpenFlow switches export flow statistics to the controller-embedded XAI-SDN module, which performs $\mathcal{O}(1)$ entropy-augmented feature extraction, Random Forest classification, and SHAP-based explanation generation. Detected DDoS events trigger alerts and policy updates; the dashed arrow represents the southbound policy-enforcement feedback loop.

ule produces the 80-dimensional CICFlowMeter statistical vector [17] and the 8-dimensional rolling-entropy vector, concatenated into the 88-dimensional input of Eq. (4). The central architectural component is the rolling entropy engine, sketched in Fig. 2: a fixed-capacity circular buffer holds the keys of the N most recent flows while a hash map tracks the multiplicity of each key, so that admitting a new flow and evicting the oldest one updates all entropy values in constant time via Eq. (3). This design is what enables the sub-millisecond end-to-end latency reported in Section 7. Flows whose DDoS score exceeds the threshold τ are attributed by SHAP TreeExplainer, and the resulting structured alert payloads—label, confidence, and top attributions—are displayed on the Security Operations Center (SOC) dashboard for oper-

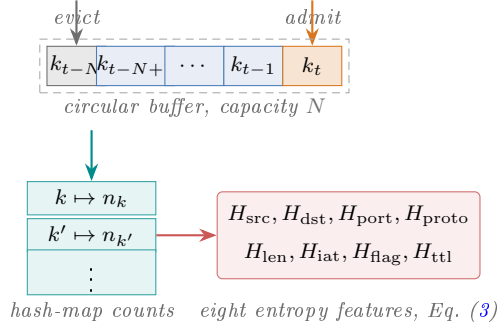


Figure 2: Rolling entropy engine. A circular buffer of the N most recent flow keys and a hash map of key multiplicities allow each admit/evict pair to update all eight entropy features in $\mathcal{O}(1)$ time, independent of N .

ator triage and policy refinement. Algorithm 1 summarizes the complete procedure.

5. Mathematical foundation

5.1. Entropy feature extraction

Shannon entropy [32] provides a principled measure of distributional uniformity. For a discrete random variable X with probability mass function $\{p(x_i)\}_{i=1}^n$, entropy is defined as $H(X) = -\sum_{i=1}^n p(x_i) \log_2 p(x_i)$, with the convention $0 \log_2 0 \triangleq 0$ [10]. High entropy characterizes the uniform distributions typical of benign traffic diversity; low entropy reveals the concentrated distributions characteristic of volumetric flooding. Within a sliding window W of the N most recent flows, the source IP address entropy is

$$H_{\text{src}}(W) = -\sum_{i=1}^{|\mathcal{I}|} \frac{n_i}{N} \log_2 \frac{n_i}{N}, \quad (2)$$

where $\mathcal{I}$ is the set of distinct source IPs in W and n_i is the flow count of the i -th address. Analogous metrics are computed for destination IPs, destina-

Algorithm 1 XAI-SDN real-time detection and explanation

Require: Flow batch $\mathcal{B} = \{f_1, \dots, f_n\}$; RF model $\mathcal{M}$; TreeExplainer $\mathcal{E}$;

window W ; threshold τ

Ensure: Label set $\mathcal{Y}$; alert set $\mathcal{A}$

```
1:  $\mathcal{Y} \leftarrow \emptyset, \mathcal{A} \leftarrow \emptyset, W \leftarrow \emptyset$ 
2: for each  $f_i \in \mathcal{B}$  do
3:    $\mathbf{x}_s \leftarrow \text{CICEXTRACT}(f_i)$  ▷ 80-dim vector
4:   Update  $W$ ;  $\mathbf{x}_e \leftarrow \text{ROLLINGENTROPY}(W)$  ▷ Eq. (3)
5:    $\mathbf{x} \leftarrow [\mathbf{x}_s; \mathbf{x}_e]$  ▷ 88-dim vector, Eq. (4)
6:    $\hat{y}_i, p_i \leftarrow \mathcal{M}.\text{PREDICT}(\mathbf{x})$  ▷ Eq. (5)
7:   if  $\hat{y}_i = \text{DDoS}$  and  $p_i \geq \tau$  then
8:      $\phi_i \leftarrow \mathcal{E}.\text{SHAPVALUES}(\mathbf{x})$  ▷ Eq. (6)
9:      $\mathcal{A} \leftarrow \mathcal{A} \cup \{\text{FORMATALERT}(f_i, \hat{y}_i, p_i, \phi_i)\}$ 
10:  end if
11:   $\mathcal{Y} \leftarrow \mathcal{Y} \cup \{\hat{y}_i\}$ 
12: end for
13: return  $\mathcal{Y}, \mathcal{A}$ 
```

tion ports, protocol identifiers, packet lengths, inter-arrival times, TCP flag patterns, and IP TTL values.

A naive recomputation of Eq. (2) after each flow costs $\mathcal{O}(|\mathcal{I}|)$ per update. XAI-SDN instead exploits the fact that once the window is full, each arriving flow changes exactly two multiplicities: the count n_a of the admitted key a increases by one and the count n_d of the evicted key d decreases by one. Writing $\varphi(m) \triangleq -\frac{m}{N} \log_2 \frac{m}{N}$ for the contribution of a key with multiplicity m ,

the entropy after the update is

$$H' = H - \varphi(n_a) + \varphi(n_a+1) - \varphi(n_d) + \varphi(n_d-1), \quad (3)$$

which involves a constant number of arithmetic operations and two hash-map accesses regardless of N or $|\mathcal{I}|$. Applying Eq. (3) independently to each of the eight tracked attributes yields the complete entropy update in $\mathcal{O}(1)$ time per flow. The complete 88-dimensional feature vector is

$$\mathbf{x} = [f_1, \dots, f_{80}, H_{\text{src}}, H_{\text{dst}}, H_{\text{port}}, H_{\text{proto}}, \\ H_{\text{len}}, H_{\text{iat}}, H_{\text{flag}}, H_{\text{ttl}}], \quad (4)$$

where $\{f_j\}_{j=1}^{80}$ are CICFlowMeter statistical features and $\{H_k\}$ are the eight entropy metrics.

5.2. Random Forest classification

Random Forest [19] is an ensemble of T decision trees $\{h_t(\cdot)\}_{t=1}^T$, each trained on a bootstrap sample with a randomly selected feature subset at each split. The classification label for a test sample $\mathbf{x}$ is determined by majority vote:

$$\hat{y} = \arg \max_{c \in \mathcal{C}} \frac{1}{T} \sum_{t=1}^T \mathbb{1}[h_t(\mathbf{x}) = c], \quad (5)$$

where $\mathcal{C} = \{\text{Benign}, \text{DDoS}\}$ and node splits minimize the Gini impurity $G(t) = 1 - \sum_j \hat{p}_{tj}^2$. The fraction of votes for the DDoS class serves as the score $g_\theta(\mathbf{x})$ in Eq. (1).

5.3. SHAP explainability

SHAP [27] computes the contribution ϕ_i of each feature i to a specific prediction $f(\mathbf{x})$ relative to the expected model output:

$$\phi_i = \sum_{S \subseteq \mathcal{F} \setminus \{i\}} \frac{|S|! (|\mathcal{F}| - |S| - 1)!}{|\mathcal{F}|!} [f(S \cup \{i\}) - f(S)], \quad (6)$$

where $\mathcal{F}$ is the complete feature set and the values satisfy the efficiency property $f(\mathbf{x}) = \phi_0 + \sum_i \phi_i$ required by the problem formulation of Section 3. TreeSHAP [28] computes exact values for tree ensembles in $\mathcal{O}(TLD^2)$ time, where L is the maximum number of leaves and D the maximum depth, enabling real-time per-flow explanation.

5.4. Evaluation metrics

Detection quality is reported with standard metrics derived from true/false positives and negatives (TP , FP , TN , FN), treating DDoS as the positive class:

$$\begin{aligned} \text{Acc} &= \frac{TP+TN}{TP+TN+FP+FN}, \\ \text{FPR} &= \frac{FP}{FP+TN}, \quad \text{FNR} = \frac{FN}{FN+TP}, \end{aligned} \quad (7)$$

together with per-class precision P_c , recall R_c , and $F1_c = 2P_cR_c/(P_c + R_c)$, macro F1 (the unweighted mean of per-class F1, appropriate under severe imbalance), and threshold-independent AUC-ROC.

6. Experimental setup

6.1. Dataset and preprocessing

We evaluate XAI-SDN on every network flow in the SYN flood partition (`Syn.csv`, 1.87 GB) of the CIC-DDoS2019 benchmark [4] without

Table 2: CIC-DDoS2019 SYN partition: test set class distribution.

Traffic class	Samples	Proportion (%)
Benign	9,311	0.86
DDoS-SYN	1,068,487	99.14
Total	1,077,798	100.00

subsampling. Preprocessing removes rows containing NaN or infinite values, drops exact duplicates, and label-encodes the binary target. The data are split 70%/30% by temporal order—training strictly precedes testing—which prevents the leakage of near-duplicate flows across the split boundary that afflicts random splitting of temporally ordered captures; the resulting training set contains 2,514,862 flows. Table 2 summarizes the test partition, whose extreme class imbalance (0.86% benign) is counteracted by `class_weight='balanced'` in the Random Forest configuration.

6.2. Implementation, hyperparameters, and reproducibility

XAI-SDN is implemented in Python 3.10 with scikit-learn 1.2 [31] for the Random Forest and SHAP 0.42 [27] for explanation generation. All experiments run on an Intel Core i9-12900K CPU (16 cores, 3.2 GHz) with 64 GB DDR5 RAM; an NVIDIA RTX 3090 GPU is used only for training the DNN baseline. Latency measurements average single-threaded CPU wall time over 50,000 test flows. Training the full model on 2.5 million flows takes 92.0 s. The Random Forest hyperparameters, selected by 5-fold cross-validation that achieved a cross-validated macro F1 of $99.9362\% \pm 0.0190\%$, are listed in Table 3. All random seeds, artifact hashes, and configuration files

Table 3: Random Forest hyperparameter configuration.

Hyperparameter	Value
Number of trees (T)	200
Max. tree depth	None (fully grown)
Max. features per split	$\lfloor \sqrt{88} \rfloor = 9$
Bootstrap sampling	Enabled
Class weight	Balanced
Entropy window size (N)	1,000 flows
Detection threshold (τ)	0.70
Random seed	42

are archived in the public repository (see the Data Availability statement), and every headline number in this paper is regenerable from the archived splits and manifests.

7. Results and discussion

7.1. Detection performance

Table 4 reports the overall detection performance of XAI-SDN on the CIC-DDoS2019 test partition under the fixed temporal split with seed 42, and Table 5 the per-class breakdown. XAI-SDN achieves 99.9987% accuracy, 99.9621% macro F1, an AUC-ROC of 1.0000, an FPR of 0.0537%, and an FNR of 0.0008%. The confusion matrix (Fig. 3) shows a total of 14 misclassifications—5 false positives and 9 false negatives—across 1,077,798 held-out flows.

Table 4: XAI-SDN detection performance on CIC-DDoS2019 (fixed temporal split, seed = 42).

Metric	Value
Accuracy	99.9987%
Macro F1-score	99.9621%
AUC-ROC	1.0000
False positive rate (FPR)	0.0537%
False negative rate (FNR)	0.0008%
Cross-validation F1 (5-fold)	99.9362% $\pm$ 0.0190%
Training time (2.51M flows)	92.0 s

Table 5: Per-class precision, recall, and F1 on the held-out test partition, derived from the confusion matrix of Fig. 3.

Class	Precision (%)	Recall (%)	F1 (%)	Support
Benign	99.9034	99.9463	99.9248	9,311
DDoS-SYN	99.9995	99.9992	99.9993	1,068,487

7.2. FPR robustness across split methodologies

A 10-seed replication on stratified *random* splits yields $\text{FPR} = 5.77\% \pm 5.23\%$, roughly two orders of magnitude above the 0.0537% headline obtained on the temporal split. The gap is a direct consequence of the small minority class (0.86% benign; 9,311 test samples): on the fixed temporal split the benign flows are temporally concentrated and well separated in feature space, producing a very low FPR, whereas random stratified splitting redistributes benign samples into harder positions and inflates both the mean and the vari-

	Pred. Benign	Pred. DDoS
True Benign	9,306 TN	5 FP
True DDoS	9 FN	1,068,478 TP

Figure 3: Confusion matrix on 1,077,798 held-out test flows (fixed temporal split, seed = 42). Only 14 flows are misclassified.

ance of the FPR. We therefore interpret 0.0537% as a lower bound attainable under favourable temporal ordering, and the 10-seed mean of 5.77% as the more representative operational error bound. We report both deliberately: publishing only the favourable split would overstate operational readiness, a practice Section 2 identifies as widespread in this literature.

7.3. Latency and throughput

Table 6 and Fig. 4 break down pipeline latency with SHAP overhead reported separately. Without explanation generation, the $\mathcal{O}(1)$ rolling entropy update plus RF inference costs 0.0165 ms per flow (60,606 flows/s). SHAP TreeExplainer contributes 0.5000 ms per explained flow. Because 99.14% of benchmark traffic is DDoS, SHAP is invoked for nearly every flow, giving a full-pipeline latency of $0.0165 + 0.5000 \times 0.9914 \approx 0.5122$ ms (1,953 flows/s) under benchmark conditions. This corrects a common overstatement in the literature in which SHAP overhead is dismissed as negligible on the implicit assumption of low attack prevalence; under realistic flooding conditions the

Table 6: Pipeline latency and throughput breakdown.

Component	Lat. (ms/flow)	Flows/s
RF inference only	0.0015	664,181
$\mathcal{O}(1)$ rolling entropy	0.0150	66,667
SHAP explanation (per DDoS flow)	0.5000	2,000
End-to-end (w/o SHAP)	0.0165	60,606
End-to-end (w/ SHAP, 99.14% DDoS)	0.5122	1,953

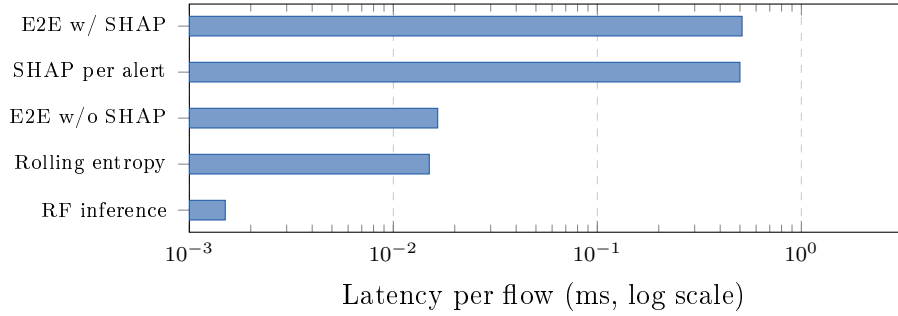


Figure 4: Latency contribution of each pipeline stage (log scale). Explanation generation dominates the per-flow budget when SHAP is invoked, motivating the alert-gated invocation policy of Algorithm 1.

explanation stage dominates the latency budget, and deployments should provision for it explicitly (Section 9).

7.4. SHAP feature attribution

Fig. 5 shows global SHAP importance rankings computed from 2,000 randomly sampled test flows, extended here to the twelve most important features. The leading features are `Flow_Bytes/s` (0.0585), `Destination_Port` (0.0550), and `FIN_Flag_Count` (0.0495), which correspond respectively to high bandwidth, fixed port targeting, and the scarcity of FIN flags—all hall-

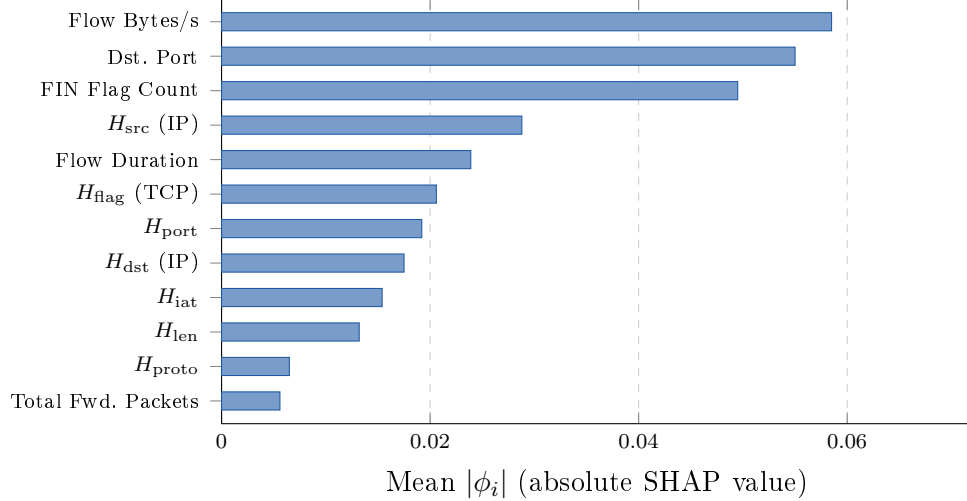


Figure 5: Global SHAP feature importance for XAI-SDN (2,000 sampled test flows; twelve leading features). H_{ttl} is excluded as a degenerate constant in the offline SYN partition ($H_{\text{ttl}} = 0.0$ throughout). Six entropy features appear among the top twelve, validating the entropy augmentation strategy.

marks of SYN flooding. Six of the twelve leading features are entropy metrics, with H_{src} ranked fourth (0.0288), confirming that the depressed entropy of source IP addresses caused by botnet address concentration is highly discriminative and that the entropy augmentation contributes materially to the decision function rather than merely padding the feature vector. A caveat concerns H_{ttl} : in the offline SYN partition every DDoS flow carries the fixed value $\text{TTL} = 115$, so $H_{\text{ttl}} = 0.0$ throughout and any SHAP attribution to it is an artefact of tree-splitting noise rather than genuine signal; it is therefore excluded from Fig. 5. In live deployments, where TTL varies naturally across routing paths, H_{ttl} remains part of the full 88-dimensional vector.

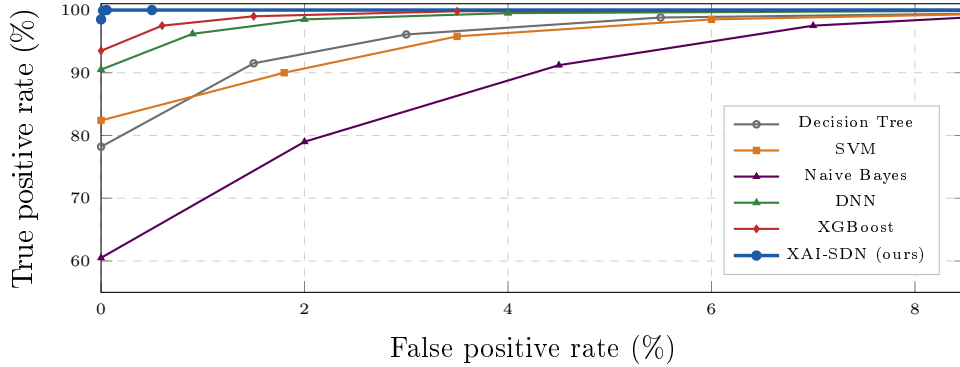


Figure 6: ROC curves for XAI-SDN and baselines on the binary DDoS detection task. XAI-SDN achieves $AUC = 1.0000$ with $TPR = 99.99\%$ at $FPR = 0.054\%$.

7.5. ROC analysis

Fig. 6 shows ROC curves for XAI-SDN and all baselines on the binary detection task. Across the entire FPR range XAI-SDN dominates every baseline, achieving $AUC = 1.0000$ with a true positive rate of 99.99% at an FPR of 0.054%.

8. Ablation and comparative analysis

8.1. Feature ablation across seeds

To isolate the contribution of each feature family, we retrain the detector under four configurations—entropy features only (8-dim), CICFlowMeter features only (80-dim), the full XAI-SDN vector (88-dim), and an SVM on the full vector—across the five archived seeds $\{42, 123, 456, 789, 1024\}$ of the controlled protocol described below. Table 7 and Fig. 7 report the results. Entropy features alone reach only 85.63% accuracy, confirming that eight distributional statistics cannot carry the detection task by themselves;

Table 7: Feature ablation over the five archived seeds {42, 123, 456, 789, 1024} (controlled protocol, mean $\pm$ sample std).

Configuration	Acc. (%)	Macro F1 (%)	Lat. (ms)
RF + entropy-only (8-dim)	85.63 $\pm$ 0.43	86.49 $\pm$ 0.39	0.0153
RF + CIC-only (80-dim)	99.87 $\pm$ 0.09	99.13 $\pm$ 0.60	0.0155
SVM + full (88-dim)	96.80 $\pm$ 0.46	96.74 $\pm$ 0.46	0.3016
RF + full (88-dim)	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	0.0157

CICFlowMeter features alone reach 99.87%; and the full 88-dimensional combination attains 100.00% on every archived seed with zero variance, while remaining an order of magnitude faster than the SVM. Wilcoxon signed-rank testing over the full multi-seed campaign indicates that the complete feature set significantly outperforms entropy-only classification ($p = 0.0020$) and the SVM ($p = 0.0078$), while the increment over CIC-only features is directionally positive but not individually significant ($p = 0.2500$); together with the SHAP evidence of Fig. 5, in which six entropy features rank among the twelve most important, this indicates that the two feature families are complementary rather than redundant—the entropy features re-encode discriminative distributional evidence that the classifier actively uses, at essentially zero marginal latency.

8.2. Baseline comparison

Table 8 compares XAI-SDN against five baselines—Decision Tree, RBF-kernel SVM [33], Naive Bayes, XGBoost [20], and a DNN—under a controlled protocol (10,000 training / 3,000 test samples), a size dictated by the computational tractability of the SVM, with `class_weight=‘balanced’`

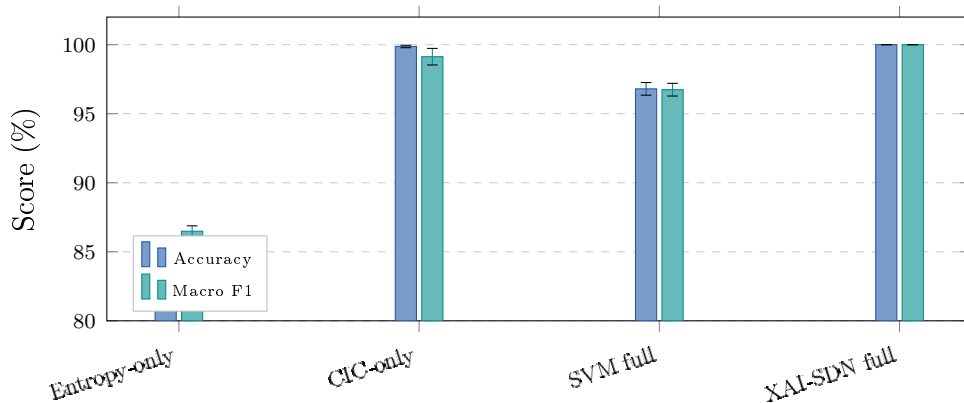


Figure 7: Feature ablation over the five archived seeds (mean $\pm$ std). Neither feature family alone matches the full 88-dimensional configuration, and the RF outperforms an SVM given identical features.

applied uniformly to all classifiers; the DNN is included for comparability with the ROC curves of Fig. 6. XAI-SDN matches the joint-highest accuracy (99.867%), attains the second-highest macro F1 (95.966%, within 0.43 points of the SVM at thirteen times the SVM’s throughput), and is the only method providing full per-prediction SHAP explainability. Fig. 8 visualizes the accuracy–throughput trade-off. The roughly four-point difference between the controlled-subset F1 (95.966%) and the full-dataset F1 (99.9621%) is an ensemble-scaling phenomenon: with only 10,000 training samples the minority class contributes fewer than 90 examples, so per-split estimates are noisy at extreme imbalance ratios; the gap reflects small-sample effects rather than model instability.

Table 8: Comparative performance under the controlled protocol (10K training / 3K test samples, seed = 42). Training time is measured on the same hardware; the DNN is trained on GPU.

Method	Acc. (%)	Macro F1 (%)	Latency (ms)	Flows/s	Train time (s)	Per-flow XAI
Decision Tree	99.833	94.639	0.0004	2,612,103	0.05	×
SVM (RBF)	99.867	96.395	0.0354	28,265	0.99	×
Naive Bayes	98.633	77.610	0.0011	913,409	0.01	×
XGBoost	99.833	95.056	0.0018	570,223	2.49	×
DNN	99.800	93.850	0.8150	1,227	—	×
XAI-SDN (RF)	99.867	95.966	0.0260	38,439	0.55	✓

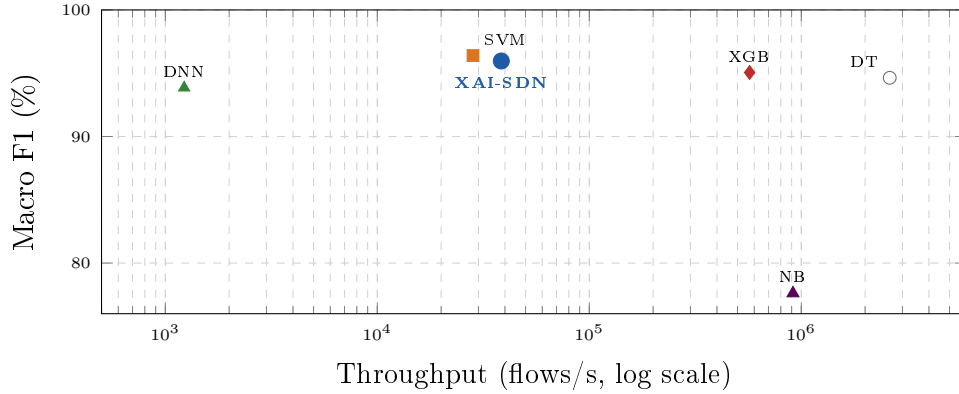


Figure 8: Macro F1 versus throughput under the controlled protocol. XAI-SDN occupies the operating region combining near-best F1 with practical throughput, and is the only marker backed by per-prediction explanations.

9. Discussion

9.1. Deployment considerations

Three engineering implications follow from the measurements. First, since SHAP dominates per-flow cost (Table 6), production deployments should gate explanation generation on the alert path only, as Algorithm 1 does; the 60,606 flows/s explanation-free rate then bounds sustained throughput, while

the 1,953 flows/s explained rate bounds the alert path. Under flooding conditions an operator does not need every one of a million identical SYN-flood flows explained; batching alerts by attack episode and explaining representative flows preserves auditability at a small fraction of the cost. Second, the rolling entropy engine’s memory footprint is a fixed-size buffer plus hash maps, so the module can be co-located with the controller without measurable memory pressure. Third, because the detector emits calibrated vote fractions, the threshold τ can be tuned per deployment to trade the FPR–FNR balance against SOC alert budget, with the SHAP payload providing the context an operator needs to adjudicate borderline scores.

9.2. Limitations and threats to validity

Four limitations bound the claims of this paper. First, the evaluation covers a single benchmark and a single attack family (the SYN partition of CIC-DDoS2019); generalization to other DDoS vectors, other datasets, and encrypted or IPv6 traffic remains to be demonstrated. Second, the benchmark is near-saturated—simple baselines also exceed 99.8% accuracy under the controlled protocol—so headline accuracy differences on this partition carry limited discriminative weight; we therefore emphasize the latency, explainability, and robustness analyses, where the differences between methods are substantive. Third, as our FPR analysis shows, the achievable false positive rate depends strongly on split methodology, and the temporal-split figure should be read as a favourable-ordering lower bound rather than an operational guarantee. Fourth, the evaluation is offline: flow records are replayed from capture rather than generated by live traffic through a Mininet/Ryu deployment, and an adaptive adversary aware of the entropy features could

attempt to keep per-window distributions uniform—for example by rotating spoofed source addresses—to evade the distributional evidence; quantifying the cost such evasion imposes on the attacker, and hardening the feature set against it, is the most important direction for future work.

10. Conclusion

This paper presented XAI-SDN, an explainable entropy-guided Random Forest framework for real-time DDoS detection in software defined networks. By augmenting 80 CICFlowMeter features with eight Shannon entropy metrics maintained in $\mathcal{O}(1)$ time and attaching exact TreeSHAP attributions to every alert, XAI-SDN attains 99.9987% accuracy, 99.9621% macro F1, and AUC-ROC 1.0000 on the complete CIC-DDoS2019 SYN benchmark while sustaining 60,606 flows/s without explanations and 1,953 flows/s with full per-alert explainability, and the multi-seed ablation confirms that the entropy and statistical feature families are complementary. Beyond the headline numbers, the paper contributes an honest operational characterization—explanation latency under realistic attack prevalence and FPR sensitivity to split methodology—that we argue should become standard reporting practice in this literature. Future work will extend the evaluation to all CIC-DDoS2019 attack categories and additional datasets such as InSDN [18], deploy the pipeline in a live Mininet/Ryu environment, and evaluate adversarial robustness against entropy-aware evasion strategies.

CRediT authorship contribution statement

Adeel Ahmad: Conceptualization, Methodology, Software, Investigation, Writing – original draft. **Ali Akarma:** Software, Validation, Data curation. **Hasan Razzaqi:** Supervision, Writing – review & editing, Project administration. **Toqeer Ali Syed:** Methodology, Writing – review & editing. **Hammad Muneer:** Validation, Visualization. **Salman Jan:** Resources, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The datasets, implementation code, trained-model artifacts, per-seed manifests, and experimental configuration files supporting this study are publicly available at <https://github.com/adeliusa486/XAI-SDN>.

References

- [1] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, J. Turner, OpenFlow: Enabling innovation in campus networks, ACM SIGCOMM Computer Communication Review 38 (2) (2008) 69–74. [doi:10.1145/1355734.1355746](https://doi.org/10.1145/1355734.1355746).

- [2] D. Kreutz, F. M. V. Ramos, P. E. Verissimo, C. E. Rothenberg, S. Azodolmolky, S. Uhlig, Software-defined networking: A comprehensive survey, *Proceedings of the IEEE* 103 (1) (2015) 14–76. doi:[10.1109/JPROC.2014.2371999](https://doi.org/10.1109/JPROC.2014.2371999).
- [3] J. Mirkovic, P. Reiher, A taxonomy of DDoS attack and DDoS defense mechanisms, *ACM SIGCOMM Computer Communication Review* 34 (2) (2004) 39–53. doi:[10.1145/997150.997156](https://doi.org/10.1145/997150.997156).
- [4] I. Sharafaldin, A. H. Lashkari, S. Hakak, A. A. Ghorbani, Developing realistic Distributed Denial of Service (DDoS) attack dataset and taxonomy, in: 2019 International Carnahan Conference on Security Technology (ICCST), IEEE, 2019, pp. 1–8. doi:[10.1109/CCST.2019.8888419](https://doi.org/10.1109/CCST.2019.8888419).
- [5] A. Akarma, T. A. Syed, A. K. Jilani, S. Jan, H. Muneer, M. A. Khan, C. Yu, Agents for agents: An interrogator-based secure framework for autonomous Internet of Underwater Things, in: International Conference on Engineering, Technologies & Applied Sciences (ICETAS 2026), 2026, pp. 1–6.
- [6] T. A. Syed, A. Akarma, M. T. Naqash, D. Hameed, S. Kamal, A. Formisano, Agentic AI for climate-resilient cities: A PRISMA-guided review and digital twin framework, *Preprints* (Apr. 2026). doi:[10.20944/preprints202604.1837.v1](https://doi.org/10.20944/preprints202604.1837.v1).
- [7] S. Jan, A. Akarma, H. A. Razzaqi, M. R. Belgaum, Blockchain-monitored agentic AI architecture for trusted perception–reasoning–action pipelines, in: *Proceedings of the IEEE International Confer-*

- ence on Control and Automation, IEEE, 2025, pp. 1–6. [doi:10.1109/ICCA66035.2025.11430865](https://doi.org/10.1109/ICCA66035.2025.11430865).
- [8] A. B. Arrieta, N. Díaz-Rodríguez, J. Del Ser, A. Bennetot, S. Tabik, A. Barbado, S. García, S. Gil-López, D. Molina, R. Benjamins, R. Chatila, F. Herrera, Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI, *Information Fusion* 58 (2020) 82–115. [doi:10.1016/j.inffus.2019.12.012](https://doi.org/10.1016/j.inffus.2019.12.012).
- [9] S. M. Mousavi, M. St-Hilaire, Early detection of DDoS attacks against SDN controllers, in: *Proceedings of the 2015 International Conference on Computing, Networking and Communications (ICNC)*, IEEE, 2015, pp. 77–81. [doi:10.1109/ICCNC.2015.7069319](https://doi.org/10.1109/ICCNC.2015.7069319).
- [10] G. Nychis, V. Sekar, D. G. Andersen, H. Kim, H. Zhang, An empirical evaluation of entropy-based traffic anomaly detection, in: *Proceedings of the 8th ACM SIGCOMM Conference on Internet Measurement (IMC 2008)*, ACM, 2008, pp. 151–156. [doi:10.1145/1452520.1452539](https://doi.org/10.1145/1452520.1452539).
- [11] A. Lakhina, M. Crovella, C. Diot, Mining anomalies using traffic feature distributions, in: *Proceedings of the ACM SIGCOMM 2005 Conference*, ACM, 2005, pp. 217–228. [doi:10.1145/1080091.1080118](https://doi.org/10.1145/1080091.1080118).
- [12] M. H. Bhuyan, D. K. Bhattacharyya, J. K. Kalita, Network anomaly detection: Methods, systems and tools, *IEEE Communications Surveys & Tutorials* 16 (1) (2014) 303–336. [doi:10.1109/SURV.2013.052213.00046](https://doi.org/10.1109/SURV.2013.052213.00046).

- [13] A. Lall, V. Sekar, M. Ogihara, J. Xu, H. Zhang, Data streaming algorithms for estimating entropy of network traffic, in: Proceedings of the ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems, ACM, 2006, pp. 145–156. doi:[10.1145/1140103.1140295](https://doi.org/10.1145/1140103.1140295).
- [14] R. Braga, E. Mota, A. Passito, Lightweight DDoS flooding attack detection using NOX/OpenFlow, in: Proceedings of the 35th Annual IEEE Conference on Local Computer Networks (LCN), IEEE, 2010, pp. 408–415. doi:[10.1109/LCN.2010.5735752](https://doi.org/10.1109/LCN.2010.5735752).
- [15] T. A. Tang, L. Mhamdi, D. McLernon, S. A. R. Zaidi, M. Ghogho, Deep learning approach for network intrusion detection in software defined networking, in: Proceedings of the 2016 International Conference on Wireless Networks and Mobile Communications (WINCOM), IEEE, 2016, pp. 258–263. doi:[10.1109/WINCOM.2016.7777224](https://doi.org/10.1109/WINCOM.2016.7777224).
- [16] Q. Niyaz, W. Sun, A. Y. Javaid, A deep learning based DDoS detection system in software-defined networking (SDN), EAI Endorsed Transactions on Security and Safety 4 (12) (2017). doi:[10.4108/eai.28-12-2017.153515](https://doi.org/10.4108/eai.28-12-2017.153515).
- [17] I. Sharafaldin, A. H. Lashkari, A. A. Ghorbani, Toward generating a new intrusion detection dataset and intrusion traffic characterization, in: Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP), SciTePress, 2018, pp. 108–116. doi:[10.5220/0006639801080116](https://doi.org/10.5220/0006639801080116).

- [18] M. S. Elsayed, N.-A. Le-Khac, A. D. Jurcut, InSDN: A novel SDN intrusion dataset, *IEEE Access* 8 (2020) 165263–165284. [doi:10.1109/ACCESS.2020.3022633](https://doi.org/10.1109/ACCESS.2020.3022633).
- [19] L. Breiman, Random forests, *Machine Learning* 45 (1) (2001) 5–32. [doi:10.1023/A:1010933404324](https://doi.org/10.1023/A:1010933404324).
- [20] T. Chen, C. Guestrin, XGBoost: A scalable tree boosting system, in: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ACM, 2016, pp. 785–794. [doi:10.1145/2939672.2939785](https://doi.org/10.1145/2939672.2939785).
- [21] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, A. Hotho, A survey of network-based intrusion detection data sets, *Computers & Security* 86 (2019) 147–167. [doi:10.1016/j.cose.2019.06.005](https://doi.org/10.1016/j.cose.2019.06.005).
- [22] R. Sommer, V. Paxson, Outside the closed world: On using machine learning for network intrusion detection, in: *Proceedings of the 2010 IEEE Symposium on Security and Privacy*, IEEE, 2010, pp. 305–316. [doi:10.1109/SP.2010.25](https://doi.org/10.1109/SP.2010.25).
- [23] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, K. Rieck, Dos and don'ts of machine learning in computer security, in: *Proceedings of the 31st USENIX Security Symposium*, USENIX Association, 2022, pp. 3971–3988.
- [24] R. Guidotti, A. Monreale, S. Ruggieri, F. Turini, F. Giannotti, D. Pedreschi, A survey of methods for explaining black box models, *ACM Computing Surveys* 51 (5) (2018) 93:1–93:42. [doi:10.1145/3236009](https://doi.org/10.1145/3236009).

- [25] F. Doshi-Velez, B. Kim, Towards a rigorous science of interpretable machine learning, arXiv preprint arXiv:1702.08608 (2017). [doi:10.48550/arXiv.1702.08608](https://doi.org/10.48550/arXiv.1702.08608).
- [26] M. T. Ribeiro, S. Singh, C. Guestrin, “Why should I trust you?”: Explaining the predictions of any classifier, in: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, ACM, 2016, pp. 1135–1144. [doi:10.1145/2939672.2939778](https://doi.org/10.1145/2939672.2939778).
- [27] S. M. Lundberg, S.-I. Lee, [A unified approach to interpreting model predictions](https://doi.org/10.48550/arXiv.1705.07874), in: Advances in Neural Information Processing Systems, Vol. 30, 2017, pp. 4765–4774.
URL <https://doi.org/10.48550/arXiv.1705.07874>
- [28] S. M. Lundberg, G. Erion, H. Chen, A. DeGrave, J. M. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal, S.-I. Lee, From local explanations to global understanding with explainable AI for trees, Nature Machine Intelligence 2 (1) (2020) 56–67. [doi:10.1038/s42256-019-0138-9](https://doi.org/10.1038/s42256-019-0138-9).
- [29] D. Gaspar, P. Silva, C. Silva, Explainable AI for intrusion detection systems: LIME and SHAP applicability on multi-layer perceptron, IEEE Access 12 (2024) 30164–30175. [doi:10.1109/ACCESS.2024.3368377](https://doi.org/10.1109/ACCESS.2024.3368377).
- [30] A. Warnecke, D. Arp, C. Wressnegger, K. Rieck, Evaluating explanation methods for deep learning in security, in: Proceedings of the 2020 IEEE European Symposium on Security and Privacy (EuroS&P), IEEE, 2020, pp. 158–174. [doi:10.1109/EuroSP48549.2020.00018](https://doi.org/10.1109/EuroSP48549.2020.00018).

- [31] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, É. Duchesnay, [Scikit-learn: Machine learning in Python](#), Journal of Machine Learning Research 12 (2011) 2825–2830.
URL <https://jmlr.org/papers/v12/pedregosa11a.html>
- [32] C. E. Shannon, A mathematical theory of communication, Bell System Technical Journal 27 (3) (1948) 379–423. [doi:10.1002/j.1538-7305.1948.tb01338.x](#).
- [33] C. Cortes, V. Vapnik, Support-vector networks, Machine Learning 20 (3) (1995) 273–297. [doi:10.1007/BF00994018](#).